

Rapport Technique Final

Audit statique et tests cibles de l'APK Android

Application analysee : com.fpe.comptenickel

Version observee : 2.12.0

Periode de travail : 24 au 25 mars 2026

Type de mission : revue statique, decompilation, tests cibles, validation de surfaces exposees

1. Resume executif

L'analyse du paquet Android fourni montre un niveau de risque globalement eleve. Les constats les plus importants sont :

- une Google API key exposee dans les ressources applicatives ;
- l'activation du cleartext traffic au niveau global ;
- le support de versions Android anciennes et vulnerables (minSdkVersion: 19) ;
- un bug logique dans la gestion RSA sur Android 4.4 a 5.1 ;
- plusieurs composants exportes inutilement ou insuffisamment proteges ;
- un chemin de code permettant un bypass TLS conditionnel via RNFetchBlob.

En revanche, le test cible mene contre l'environnement Firebase n'a pas mis en evidence

d'accès public aux donnees : la base Realtime Database, l'authentification anonyme et

le bucket Storage repondent par des refus d'accès.

Conclusion executive :

- le binaire revele une surface d'attaque significative ;
- l'exposition directe de donnees Firebase n'a pas ete demontree ;
- les priorites de remediations portent d'abord sur les secrets, le transport reseau, la compatibilite legacy et les composants exportes.

2. Contexte et objectifs

L'objectif de la mission etait de produire un rapport technique consolide a partir des

travaux menes sur les Stories 1 a 9 :

- verification de l'APK fourni ;
- analyse automatisee MobSF ;
- extraction des fichiers structurants ;
- decompilation avec Apktool ;
- revue des secrets et constantes sensibles ;
- revue du code Smali ;
- recompilation de l'APK ;

- signature d'une version modifiée ;
- vérification de l'exposition Firebase.

Ce rapport rassemble les observations, les preuves, l'évaluation du risque et les recommandations de sécurité prioritaires.

3. Perimetre et methodologie

3.1 Perimetre

Les artefacts suivants ont été exploités :

- app.apk
- us-1/verification-hash.md
- us-2/mobsf_resume.md
- us-2/Static Analysis.csv
- us-3/extraction-fichiers.md
- us-4/app_src/
- us-5/strings_findings.md
- us-6/code_findings.md
- us-7/recompilation-apk.md
- us-8/signature-apk.md
- us-9/firebase-test.md

3.2 Demarche

- vérification d'intégrité du binaire fourni ;
- scan automatisé pour obtenir une première cartographie du risque ;
- extraction et décompilation pour lire le manifest, les ressources et le Smali ;
- revue manuelle des secrets, des configurations et des composants exportés ;
- recompilation et resignature pour évaluer la faisabilité d'un scénario de repackaging ;
- test cible sur Firebase afin de distinguer exposition de secrets et exploitation réelle.

3.3 Limites

- analyse majoritairement statique ;
- pas d'installation sur terminal physique ou émulateur documentée ici ;
- pas de trafic réseau observé en conditions réelles ;
- l'option trusty de RNFetchBlob est prouvée dans le binaire, mais son activation effective n'a pas été retrouvée dans le bundle JavaScript extrait.

4. Synthèse des artefacts et constats techniques

4.1 Identification de l'application

- package : com.fpe.comptenickel
- version : 2.12.0
- hash SHA256 de l'APK fourni :
f01f08c8b6e2fe4612e81bc7e3a3ba9440dad0d7a962f4e67640390dd721d528

- minSdkVersion observe : 19
- targetSdkVersion observe : 28

4.2 Reversibilite du binaire

La mission a confirme que le paquet pouvait etre :

- decompile avec Apktool ;
- recompile en us-7/rebuilt.apk ;
- signe de nouveau en us-8/signed.apk.

Ce point ne constitue pas a lui seul une vulnerabilite, mais il confirme qu'un acteur

malveillant peut modifier le code, reconstruire une version alteree et la redistribuer

apres resignature avec un certificat tiers.

5. Evaluation des risques

5.1 Niveau global

Niveau de risque global retenu : HAUT

Justification :

- presence d'un secret exploitable cote client ;
- transport reseau insuffisamment contraint ;
- compatibilite avec des versions Android obsoletes ;
- logique cryptographique defectueuse sur une partie du parc supporte ;
- exposition de composants et de donnees inter-app.

5.2 Repartition synthetique

Severite	Nombre	Remarques
CRITIQUE	1	Exposition d'une Google API key
HAUT	3	Cleartext traffic, support Android obsolete, bug RSA
MOYEN	3	Provider exporte, receiver spoofable, bypass TLS conditionnel
FAIBLE	2	Endpoints hardcodes, residu de configuration dev
INFO	2	Firebase non exploitable publiquement, usage du keystore Android

6. Findings details

F1. Google API key exposee dans les ressources (CRITIQUE)

Source : us-5/strings_findings.md

Preuve

- champ observe : google_api_key
- valeur observee : AIzaSyBTgzttvImsUfMWDa41PCrDWAj7dmyIDhUg

Analyse

La cle est embarquee dans l'application et recouvrable apres simple decompilation. Une API key Google n'est pas un secret fort au sens strict si elle est correctement restreinte, mais son exposition facilite la reconnaissance, l'abus de quota et l'exploitation de services mal cloisonnes.

Impact

- utilisation frauduleuse de services Google si restrictions insuffisantes ;
- consommation de quota ;
- augmentation de la surface de reconnaissance.

Recommandation

- restreindre la cle aux services strictement necessaires ;
- limiter par package Android et empreinte de signature si possible ;
- faire tourner/renouveler la cle si son historique d'usage est incertain.

F2. Cleartext traffic autorise globalement (HAUT)

Sources : us-2/mobsf_resume.md, us-6/code_findings.md

Preuve

- android:usesCleartextTraffic="true"

Analyse

Le manifest autorise globalement le trafic HTTP non chiffre. Ce choix affaiblit la posture reseau de l'application, surtout sur un contexte bancaire ou para-bancaire.

Impact

- interception et modification potentielles d'un flux HTTP ;
- exposition accrue en cas de redirection ou de mauvaise configuration distante ;
- reduction de la garantie de confidentialite sur le transport.

Recommandation

- positionner usesCleartextTraffic a false ;
- definir un networkSecurityConfig explicite si une exception technique est imposee.

F3. Support de versions Android obsoletes (HAUT)

Sources : us-2/mobsf_resume.md, us-4/app_src/apktool.yml

Preuves

- minSdkVersion: 19
- targetSdkVersion: 28

Analyse

Le binaire reste compatible avec Android 4.4 et vise un niveau de SDK ancien. Cette decision elargit le perimetre d'execution a des plateformes qui ne recoivent

plus de correctifs et oblige l'application a conserver des branches de code legacy.

Impact

- exposition a un parc d'OS connus pour etre fragiles ;
- maintien de comportements anciens moins protecteurs ;
- augmentation de la complexite de securisation et de test.

Recommandation

- relever le minSdkVersion ;
- aligner le targetSdkVersion sur une version recente ;
- supprimer les branches legacy devenues inutiles.

F4. Bug logique RSA sur Android 4.4 a 5.1 (HAUT)

Source : us-6/code_findings.md

Preuve

La branche API < 23 genere la cle sous keyTag, alors que sign() tente ensuite de la relire sous un alias reconstruit de type keyTag_suffixe.

Analyse

Ce default n'est pas un simple probleme de style : il peut rendre indisponible le parcours de signature sur les appareils API 19 a 22, pourtant explicitement supportes par l'application.

Impact

- echec de signature ou d'enrolement ;
- cles potentiellement orphelines dans le keystore ;
- comportement non fiable sur les appareils legacy.

Recommandation

- unifier strictement l'alias utilise dans generateKeys() et sign() ;
- tester la fonctionnalite sur API 19 a 22 ou retirer ce support.

F5. Content Provider exporte exposant un identifiant device Pushwoosh (MOYEN)

Source : us-6/code_findings.md

Preuves

- provider exporte :
com.fpe.comptenickel.PushwooshSharedDataProvider
- route observee :
content://com.fpe.comptenickel.PushwooshSharedDataProvider/hwid

Analyse

Le provider repond sur une route permettant de recuperer un identifiant de device associe au SDK Pushwoosh. La logique de comparaison presente dans le code ressemble plus a une obscurcification qu'a un veritable mecanisme d'autorisation.

Impact

- fuite locale d'un identifiant inter-app ;
- surface supplementaire pour le tracking ou l'abus d'integration push.

Recommandation

- passer le provider en non exporte s'il n'existe pas de besoin fonctionnel avoue ;
- sinon, ajouter une permission applicative dediee.

F6. BroadcastReceiver INSTALL_REFERRER exporte et spoofable (MOYEN)

Source : us-6/code_findings.md

Preuve

Le receiver `com.learnium.RNDeviceInfo.RNDeviceReceiver` est exporte et ecrit la valeur recue dans les `SharedPreferences` sans mecanisme visible d'authentification.

Analyse

Sur les versions Android anciennes supportees par l'application, ce type de broadcast

peut etre sujet a la falsification. Cela affecte la fiabilite des donnees d'attribution.

Impact

- pollution des donnees marketing et attribution ;
- risque de logique basee sur une information non fiable.

Recommandation

- supprimer ce receiver si inutile ;
- sinon, reduire son exposition et traiter la donnee comme non fiable.

F7. Bypass TLS dormant via l'option trusty de RNFetchBlob (MOYEN)

Source : us-6/code_findings.md

Preuves

- presence d'un flag `trusty` dans la configuration `RNFetchBlob` ;
- `checkServerTrusted()` vide ;
- `HostnameVerifier.verify()` retourne toujours `true`.

Analyse

Le binaire embarque un chemin de code capable de desactiver les verifications TLS. L'activation n'a pas ete observee dans le bundle JavaScript, mais la capacite existe

et pourrait etre reintroduite ou activee ulterieurement.

Impact

- MITM complet si l'option est activee ;
- perte de validation du certificat et du nom d'hote.

Recommandation

- supprimer ce mode unsafe des builds de production ;
- interdire sa compilation hors environnement de test strictement controle.

F8. Endpoints et identifiants techniques hardcodes (FAIBLE)

Sources : us-5/strings_findings.md, us-6/code_findings.md

Preuves

- endpoints api.nickel.eu exposes ;
- identifiants Pushwoosh, GCM et Crashlytics exposes ;
- marqueurs ENVIRONMENT=production et BUILD_TYPE=release.

Analyse

Ces informations ne suffisent pas a compromettre l'application, mais elles simplifient

la reconnaissance et donnent une cartographie claire de l'ecosysteme technique.

Recommandation

- considerer toute constante embarquee comme publique ;
- ne jamais fonder une protection sur la discretion du client mobile.

F9. Residus de configuration developpeur dans le manifest (FAIBLE)

Source : us-6/code_findings.md

Preuve

- presence de DevSettingsActivity dans le manifest ;
- DEBUG=false et support developpeur desactive.

Analyse

Le risque direct semble limite dans l'etat observe, mais cette declaration ne devrait

pas apparaitre dans un manifest de production.

Recommandation

- retirer les composants de dev inutiles des builds de production.

7. Verifications rassurantes

V1. Firebase non accessible publiquement

Source : us-9/firebase-test.md

Les tests menes sur :

- la Realtime Database ;
- l'API d'authentification Firebase ;
- le bucket Google Cloud Storage ;

ont retourne des refus d'accès (Permission denied, ADMIN_ONLY_OPERATION, Anonymous caller does not have storage.objects.list access).

Conclusion

La presence d'identifiants Firebase dans strings.xml ne s'est pas traduite, dans le perimetre de ce test, par une fuite de donnees publiquement exploitable.

V2. Usage du keystore Android pour RSA

Source : us-6/code_findings.md

Le code analyse utilise AndroidKeyStore, une taille de cle de 2048 bits et SHA512withRSA. Le probleme releve concerne la logique d'alias sur API anciennes, pas la presence d'une cle privree hardcodee dans le binaire.

8. Recommandations prioritaires

- Restreindre, controler ou faire tourner la Google API key exposee.
- Desactiver le cleartext traffic et imposer une politique TLS explicite.
- Relever le niveau minimal Android supporte et mettre a jour le target SDK.
- Corriger le bug d'alias RSA sur la branche API < 23.
- Fermer ou proteger les composants exportes (PushwooshSharedDataProvider, RNDeviceReceiver).
- Supprimer completement le mode trusty de RNFetchBlob en production.
- Nettoyer les artefacts de developpement et documenter une baseline de securite mobile.

9. Conclusion

L'application analysee presente un profil typique d'application mobile riche mais insuffisamment durcie : exposition de secrets techniques, support legacy tres large, configuration reseau permissive et plusieurs surfaces d'attaque annexes.

Le point positif majeur est que l'exposition Firebase n'a pas abouti a une lecture publique des donnees lors des tests effectues. En revanche, cela ne compense pas les faiblesses du binaire lui-meme, qui justifient un plan de remediation rapide.

Le rapport final retient donc :

- une exposition reelle de secrets et de mauvaises pratiques ;
- une exposition potentielle forte au MITM et au repackaging ;

- un besoin prioritaire de durcissement applicatif avant toute diffusion sensible.

10. Livrables et traces exploitees

- us-1/verification-hash.md
- us-2/mobsf_resume.md
- us-2/Static Analysis.csv
- us-3/extraction-fichiers.md
- us-4/decompilation-apktool.md
- us-5/strings_findings.md
- us-6/code_findings.md
- us-7/rebuilt.apk
- us-7/recompilation-apk.md
- us-8/signed.apk
- us-8/signature-apk.md
- us-9/firebase-test.md